

1. Klasifikujte příslušnost jazyků L_1 a L_2 do tříd REC (rekurzivní) a RE (rekurzivně vyčíslitelné) a dokažte (ne-)příslušnost do těchto tříd.

- $L_1 = \{\langle M \rangle \# \langle G \rangle \mid M \text{ je TS a } G \text{ je gramatika typu 2, t.ž. } \exists w \in \{0, 1\}^* : w \in L(M) \wedge w \notin L(G)\}$
- $L_2 = \{\langle M \rangle \# \langle G \rangle \mid M \text{ je TS a } G \text{ je gramatika typu 2, t.ž. } \exists w \in \{0, 1\}^* : w \notin L(M) \wedge w \in L(G)\}$

1.5 body

CFG = Context-Free Grammar | ATM = Acceptance Problem for Turing Machines | CYK = Cocke–Younger–Kasami algoritmus

1 Jazyk L_1

1.1 $L_1 \in RE$

Tvrzení 1. *Jazyk L_1 je rekurzivně vyčíslitelný.*

Důkaz. Sestrojíme nedeterministický algoritmus, který akceptuje právě vstupy z L_1 :

1. Dovetailingem generujeme všechna slova $w \in \{0, 1\}^*$.
2. Pro každé w paralelně:
 - simulujeme TS M na vstupu w ,
 - rozhodnutelným testem (např. CYK) zjistíme, zda $w \in L(G)$.
3. Pokud nalezneme slovo w , pro které TS M přijme a zároveň $w \notin L(G)$, akceptujeme.

Jazyk L_1 je tedy v RE . □

1.2 $L_1 \notin REC$

Tvrzení 2. *Jazyk L_1 není rozhodnutelný.*

Důkaz. Redukujeme z problému

$$ATM = \{\langle M_0, w_0 \rangle \mid M_0 \text{ akceptuje } w_0\}.$$

Pro libovolné $\langle M_0, w_0 \rangle$ sestrojíme:

- TS M' , který ignoruje vstup a simuluje $M_0(w_0)$,
- CFG G , která generuje prázdný jazyk.

Pak:

$$\langle M_0, w_0 \rangle \in ATM \iff \langle M' \rangle \# \langle G \rangle \in L_1.$$

Tím máme redukci $ATM \leq_m L_1$, a protože $ATM \notin REC$, platí i $L_1 \notin REC$. □

Závěr.

$$L_1 \in RE, \quad L_1 \notin REC.$$


2 Jazyk L_2

2.1 $L_2 \notin \text{RE}$

Tvrzení 3. *Jazyk L_2 není rekurzivně vyčíslitelný.*

Důkaz. Redukujeme z komplementu problému přijímání:

$$\text{ATM}^c = \{\langle M_0, w_0 \rangle \mid M_0 \text{ neakceptuje } w_0\}.$$

Pro vstup $\langle M_0, w_0 \rangle$ sestrojíme:

- CFG G , která generuje jediné slovo s ,
- TS M' , který:
 - zamítne každý vstup kromě slova s ,
 - na vstupu s simuluje $M_0(w_0)$.

Pak:

$$\langle M_0, w_0 \rangle \in \text{ATM}^c \iff \langle M' \rangle \# \langle G \rangle \in L_2.$$

Z toho plyne $L_2 \notin \text{RE}$.

□

15

2. Pro neorientovaný graf definujme problém obarvení grafu k -barvami s j -chybami následovně: Nechť $G = (V, E)$ je neorientovaný graf, $k \in \mathbb{N}, k > 0$ počet barev, $j \in \mathbb{N}, j > 0$ počet povolených chyb. Graf G je obarvitelný k barvami s nejvýše j chybami pokud:

- (a) existuje funkce $c : V \rightarrow \langle 1, k \rangle$ (každý vrchol grafu obarvíme nějakou barvou)
- (b) $\exists E' \subseteq E : |E| - |E'| \leq j$ (existuje podmnožina hran původního grafu, která vznikla odstraněním maximálně j hran z původního grafu)
- (c) $\forall \{a, b\} \in E' : c(a) \neq c(b)$ (každá hrana z E' propojuje vrcholy obarvené jinou barvou)

Dokažte, že problém obarvení s chybami je NP-úplný.

(Pozn: Pomůže Vám NP-úplnost některého z problémů uvedených zde:

https://en.wikipedia.org/wiki/NP-completeness#NP-complete_problems

v odstavci „NP-complete problems“.)

1.5 body

Triviální a polynomiální případy

- $k = 1$. V tomto případě je graf obarvitelný jednou barvou s nejvýše j chybami $\iff$ počet hran $|E|$ splňuje $|E| \leq j$. To lze rozhodnout v čase $O(1)$, tj. problém je v P.
- **Velké j .** Pokud $j \geq |E|$, pak lze vždy zvolit libovolné obarvení a odstranit všechny hrany — instance je triviálně kladná.

Proto smysl má zkoumat složitost pouze pro $k \geq 2$ a „rozumná“ j .

Problém je v NP

Věta 1. *Problém k -OBARVENÍ-S-J-CHYBAMI je v třídě NP.*

Důkaz. Jako certifikát postačí dvojice (c, E') , kde $c : V \rightarrow \{1, \dots, k\}$ je přiřazení barev a $E' \subseteq E$ je množina hran, jejichž koncové vrcholy musí mít různé barvy. Verifikátor v polynomiálním čase:

- spočítá $|E'|$ a ověří, že $|E| - |E'| \leq j$,
- pro každou hranu $\{a, b\} \in E'$ ověří, že $c(a) \neq c(b)$.

Obě tyto kontroly lze provést v čase poly $(|V| + |E| + \log k + \log j)$. Tudíž problém je v NP. □

NP-hard pro $k \geq 2$

Věta 2. *Pokud $k \geq 2$ (k, j jsou součástí vstupu), pak problém k -OBARVENÍ-S-J-CHYBAMI je NP-úplný.*

Důkaz. Ukážeme, že problém je NP-hard pro $k = 2$ a $k \geq 3$, a protože je v NP, je NP-úplný.

Případ $k \geq 3$

Pro $k \geq 3$ je snadné provést redukcí z klasického k -COLORABILITY, který je NP-úplný: Daný graf G z k -COLORABILITY převedeme na instanci $(G, k, j \neq 0)$. Pak:

G je k -obarvitelný $\iff (G, k, 0)$ je kladná instance k -obarvení s 0 chybami.

Tedy NP-hard.

Případ $k = 2$

Pro $k = 2$ provedeme redukci z NP-úplného problému MAX-CUT:

- Vstup (G, t) do Max-Cut převádíme na instanci $(G, k = 2, j = |E| - t)$.
- Barvení grafu dvěma barvami je ekvivalentní rozdělení vrcholů na dvě části (řezu). Hrany spojující různobarevné vrcholy tvoří řez; hrany spojující stejné barvy jsou „chyby“.
- Počet chyb je $|E| - \text{velikost řezu}$. Požadavek „nejvýše j chyb“ znamená „řez o velikosti $\geq t$ “.
- Tedy: existuje řez velikosti $\geq t$ v $G \iff$ existuje barvení s nejvýše $j = |E| - t$ chybami.

Proto je rozhodnutí o 2-obarvení s nejvýše j chybami NP-hard.

Kombinací s tím, že problém je v NP, dostáváme, že pro $k = 2$ je problém NP-úplný. $\square$

Shrnutí

- Pro $k = 1$ nebo $j \geq |E|$ je úloha polynomiální — jde o triviální / rozhodnutelné případy.
- Pro $k \geq 2$ a obecné j je problém NP-úplný.
- Certifikátem pro NP je přiřazení barev + množina hran bez konfliktů; NP-nesnadnost plyne buď z klasického k -coloringu (pro $k \geq 3$), nebo z MAX-CUT (pro $k = 2$).

3. Mějme teorii T se signaturou $\langle \{Cat_{/0}, Fish_{/0}, Trex_{/0}\}, \{eats_{/2}, =_{/2}\} \rangle$ (= je standardní rovnost) se speciálními axiomy

$$\begin{aligned} \forall x(x = Fish \vee x = Cat \vee x = Trex) \\ \forall x \forall y(eats(x, y) \leftrightarrow \neg eats(y, x)) \\ \forall x(x = Trex \vee eats(Trex, x)) \end{aligned}$$

- i) Rozhodněte a stručně zdůvodněte, zda T je (a) bezesporná, (b) úplná a (c) rozhodnutelná (tj. množina důsledků T je rozhodnutelná).
ii) Rozhodněte a stručně zdůvodněte, zda může být Peanova aritmetika T_{PA} redukovatelná na T (tj. zda může existovat vyčíslitelná redukce f , která každou větu φ Peanovy aritmetiky převede na větu $f(\varphi)$ jazyka teorie T takovou, že $T_{PA} \models \varphi \iff T \models f(\varphi)$).

1 bodů

(i) Bezespornost, úplnost a rozhodnutelnost

Bezespornost

Lemma 1. *Teorie T je sporná.*

Důkaz. Z axiomu (A2) instancujeme $x = y$:

$$\forall x(eats(x, x) \leftrightarrow \neg eats(x, x)).$$

Zvolíme např. $x = Cat$:

$$eats(Cat, Cat) \leftrightarrow \neg eats(Cat, Cat),$$

což je kontradikce. Proto T je sporná. □

Úplnost

Lemma 2. *Teorie T je úplná.*

Důkaz. Každá sporná teorie odvozuje libovolnou formuli. Tudíž pro každou větu φ platí $T \vdash \varphi$. □

Rozhodnutelnost množiny důsledků

Lemma 3. *Množina důsledků T je rozhodnutelná.*

Důkaz. Protože $T \vdash \varphi$ pro každou větu φ , rozhodnutí, zda $T \vdash \varphi$, je triviálně algoritmicky možné (vždy ANO). Tedy množina důsledků je rozhodnutelná. □

(ii) Nemožnost redukce Peanovy aritmetiky na T

Věta 1. *Neexistuje vyčíslitelná redukce*

$$f : \{\text{věty TPA}\} \rightarrow \{\text{věty jazyka } T\}$$

taková, že pro každou větu φ Peanovy aritmetiky platí

$$TPA \models \varphi \quad \Leftrightarrow \quad T \models f(\varphi).$$

Důkaz. Argument: - Pokud by taková vyčíslitelná redukce f existovala, pak by rozhodnutí, zda TPA odvozuje φ , se dalo provést následovně: 1. Spočítat $f(\varphi)$ (algoritmicky možné). 2. Zjistit, zda $T \models f(\varphi)$.

- Ale T je sporná $T \models \psi$ pro každou formuli ψ . Tudíž pravá strana je vždy pravdivá, nezávisle na φ .

- To by znamenalo, že pro libovolnou φ platí $TPA \models \varphi \iff$ pravda, tj. že TPA odvozuje všechny věty. To je nepravda (TPA není triviální), takže redukce f nemůže existovat.

- Alternativně: existence takové redukce by implikovala rozhodnutelnost množiny důsledků TPA, což je v rozporu s Gödelovým větami o neúplnosti a rozhodnutelnosti. $\square$

Shrnutí

- T je sporná, a tedy (triviálně) úplná.
- Množina důsledků T je rozhodnutelná (triviálně, protože odvozuje všechny formule).
- Peanova aritmetika TPA se na T vyčíslitelně redukovat nemůže.

4. Uvažujeme pole celých čísel proměnné délky, jehož datová struktura obsahuje proměnné (i) *arr*—ukazatel na začátek pole, (ii) *size*—velikost alokovaného prostoru, (iii) *used*—počet využitých položek od začátku. Pro práci s polem máme následující funkce:

- *array_init*: alokuje pole velikosti 10 čísel (*malloc*) a nastaví *size* a *used*.
- *insert*: v případě, že je volné místo, vloží prvek *val* na konec pole a zvýší počet obsazených pozic o 1. V případě, že volné místo není (*size* = *used*), tak alokuje nové pole čísel *tmp* o velikosti *size* * 2, zkopíruje obsah původního pole do nového, dealokuje staré pole (*free*) a ukazatel na nové pole uloží do proměnné *arr*. Poté vloží prvek *val* do zvětšeného pole a obsah pole nechá seřadit funkcí *sort*.
- *sort*: seřadí pole pomocí modifikovaného bubble sortu.

(i) Analyzujte a zdůvodněte složitost jednoho volání *insert* v nejhorším případě.

(ii) S využitím amortizované složitosti analyzujte a zdůvodněte složitost posloupnosti *n* operací *insert* aplikované na pole inicializované funkcí *array_init*.

Poznámky:

- Předpokládejme uniformní cenové kritérium, kde každý řádek má cenu 1 (s výjimkou volání funkce *sort*).
- Funkce *swap(a, b)* prohodí položky hodnoty proměnných *a, b*.

```

1 Struct array
2   int arr[];
3   int size, used;
4 Function array_init(array a)
5   a.arr := malloc(10);
6   a.size := 10;
7   a.used := 0;
8 Function insert(int val, array a)
9   resize := false;
10  if a.used = a.size then
11    resize := true;
12    tmp[] := malloc(a.size * 2);
13    for (i := 0; i < size; i++) do
14      tmp[i] = a.arr[i];
15    free(a.arr);
16    a.arr := tmp;
17    a.size := a.size * 2;
18  a.arr[a.used] := val;
19  a.used := a.used + 1;
20  if resize then
21    sort(a);
22 Function sort(array a)
23   sorted := false;
24   while ¬sorted do
25     sorted := true;
26     for (i := a.used - 1; i > 0; i = i - 1) do
27       if a.arr[i] < a.arr[i - 1] then
28         swap(a.arr[i], a.arr[i - 1]);
29       sorted := false;
```

2 body

(i) Nejhorší případ jednoho volání `insert`

Nechť před voláním platí `used = size = s`; tedy vkládání vyvolá *resize*.

Při zvětšení se provede:

- alokace nového pole délky $2s$,
- cyklus kopírování o délce s , tj. $\Theta(s)$,
- vložení prvku na pozici s ,
- volání `sort(a)`, protože došlo k rozšíření.

Funkce `sort` je modifikovaný bubble sort. Pokud je pole v nejhorším možném stavu (z hlediska počtu výměn), trvá:

$$\Theta((s+1)^2) = \Theta(s^2).$$

Celková cena operace je tedy:

$$\Theta(s) + \Theta(s^2) = \Theta(s^2).$$

Výsledek: Nejhorší případ jednoho volání `insert` je časové složitosti

$$\boxed{\Theta(s^2)},$$

kde $s = \text{size}$ před zvětšením.

(ii) Amortizovaná složitost posloupnosti n operací insert

Nechť počáteční kapacita je $C_0 = 10$. Při každém rozšíření se kapacita zdvojnásobí:

$$C_0 = 10, C_1 = 20, C_2 = 40, \dots, C_t = 10 \cdot 2^t.$$

Počet zvětšení kapacity je:

$$R = \left\lceil \log_2 \left(\frac{n}{10} \right) \right\rceil.$$

Náklady operací bez zvětšení

Tyto operace stojí $\Theta(1)$ každá, tedy celkem $\Theta(n)$.

Náklady operací se zvětšením

Každé zvětšení z kapacity C_i vyvolá řazení pole o délce $C_i + 1$. Cena řazení je:

$$\Theta(C_i^2).$$

Celkové náklady řazení v průběhu všech zvětšení jsou:

$$\sum_{i=0}^{R-1} \Theta(C_i^2) = \Theta \left(\sum_{i=0}^{R-1} (10 \cdot 2^i)^2 \right) = \Theta \left(\sum_{i=0}^{R-1} 100 \cdot 4^i \right).$$

Geometrická řada $\sum 4^i$ má součet $\Theta(4^R)$, tedy:

$$\sum_{i=0}^{R-1} C_i^2 = \Theta(4^R) = \Theta((10 \cdot 2^R)^2) = \Theta(n^2).$$

Dominujícím nákladem je řazení při zvětšení.

Celkový čas posloupnosti

$$T(n) = \Theta(n) + \Theta(n^2) = \Theta(n^2).$$

Amortizovaná cena jedné operace

$$\frac{T(n)}{n} = \Theta(n).$$

Výsledek: Amortizovaná cena jednoho volání insert je:

$$\boxed{\Theta(n)}.$$



5. Nechť $G = (V, E)$ je orientovaný graf. Definujeme sled jako

$sled(v_1, v_n) \Leftrightarrow \exists$ posloupnost $v_1 e_1 v_2 e_2 \dots e_{n-1} v_n : \forall 0 < i < n : e_i = (v_i, v_{i+1})$ a $e_i \in E$. Uzly a hrany se mohou v posloupnosti opakovat.

Pro graf $G = (V, E)$, uzly $v_s, v_f \in V$ a hranu $e \in E$ uvažujme problém existence sledu $sled(v_s, v_f)$, který obsahuje hranu e .

Uvažujme funkci *check*, která prohledá všechny sledy do délky $2 * |V|$ a zjistí, zda takový sled existuje.

- Analyzujte časovou složitost funkce *check* v nejlepším případě
- Analyzujte časovou složitost funkce *check* v nejhorším případě.
- Navrhněte funkci *check_opt*, která bude dávat stejný výsledek jako funkce *check*, ale bude mít lepší asymptotickou složitost v nejhorším případě.
- Analyzujte časovou složitost funkce *check_opt* v nejhorším případě.

```
1 Function check((V, E), vs, vf, e)
2   s := empty_stack();
3   s.push((vs, false, 0));
4   while stack_noempty(s) do
5     v, visited, depth := s.pop();
6     foreach w ∈ succ(v) do
7       visitednew := visited;
8       if (v, w) = e then
9         visitednew := true;
10      if visitednew ∧ w = vf then
11        return true;
12      if depth < 2|V| then
13        s.push((w, visitednew, depth + 1))
14  return false;
```

Poznámky:

- Předpokládejme uniformní cenové kritérium, kde každý řádek má cenu 1.
- Funkce *succ*(v) vrací všechny následníky vrcholu v v $O(1)$.
- Analýzu proveďte vzhledem k počtu vrcholů $|V|$. V případě *check_opt* můžete analyzovat vyhledem k počtu vrcholů $|V|$ a hran $|E|$.
- Do zásobníku vkládáme trojice $(v, visited, depth)$, kde v je vrchol, *visited* udává, jestli jsme prošli hranou e a *depth* aktuální délku zanoření (délku sledu).

2 body

Nechť $n = |V|$.

(1) Časová složitost check v nejlepším případě

V nejlepším případě se hrana e objeví hned při prvním rozvinutí následníků vrcholu v_s . Funkce okamžitě vrátí `true`.

$$\boxed{\Theta(1)}.$$

(2) Časová složitost check v nejhorším případě

Funkce generuje všechny sledy délky až $2n$. Při maximálním stupni Δ je počet navštívených stavů

$$1 + \Delta + \Delta^2 + \dots + \Delta^{2n} = \Theta(\Delta^{2n}).$$

V nejhorším grafu platí $\Delta = n - 1$, takže

$$\boxed{O((n-1)^{2n}) = 2^{\Theta(n \log n)}}.$$

Složitost je exponenciální v n .

(3) Návrh optimalizované funkce check_opt

Sled obsahující hranu $e = (a, b)$ a vedoucí z v_s do v_f existuje právě tehdy, když:

$$v_s \rightsquigarrow a \quad \text{a zároveň} \quad b \rightsquigarrow v_f,$$

kde $\rightsquigarrow$ označuje dosažitelnost.

Proto stačí provést dvě prohledání grafu:

```
Function check_opt((V,E), vs, vf, e=(a,b)):  
    if not reachable(vs, a): return false  
    if reachable(b, vf): return true  
    else: return false
```

kde `reachable` je BFS/DFS.

2

(4) Časová složitost `check_opt`

Použijeme dvě BFS/DFS:

$$O(|V| + |E|) + O(|V| + |E|) = O(|V| + |E|).$$

$$\boxed{O(|V| + |E|)}.$$